

به نام خدا

عنوان پروژه: MCP + Llama ۳ + Confluence

عنوان سند: گزارش فنی اجرای Llama ۳ و بررسی اتصال به Confluence

نسخه های سند:

| عنوان                              | تهیه کننده       | تاریخ تنظیم | شماره نسخه |
|------------------------------------|------------------|-------------|------------|
| راه اندازی و پیاده سازی MCP server | احسان جلالی رومی | ۱۴۰۵/۰۷/۱۲  | ۱          |

شناسنامه سند:

| عنوان پروژه | MCP Server                         |
|-------------|------------------------------------|
| عنوان سند   | راه اندازی و پیاده سازی MCP server |
| کد سند      | ۱۴۰۵۰۷۱۲-mcp-server-v۱             |
| طبقه بندی   | فاقد طبقه بندی                     |
| شماره سند   | ۱                                  |
| تعداد صفحات | ۱۱                                 |

## ۱. خلاصه مدیریتی

هدف این گزارش ارائه وضعیت سه بخش اصلی پروژه است: پیاده سازی یک سرور MCP محلی، اجرای Llama ۳ به صورت Local و بررسی امکان اتصال سامانه به Confluence داخلی شرکت.

ابتدا پکیج mcp پایتون نصب شد و سپس سه تابع پایتونی به عنوان ابزار دستی و محلی تعریف شدند. سپس در بخش کلاینت ابزارها فراخوانی و اجرا شدند که کل پایپ لاین سیستم موفقیت آمیز بود.

در مرحله بعدی Llama ۳,۲ با Ollama روی سیستم محلی اجرا شد و از طریق LangChain در قالب Agent مورد استفاده قرار گرفت. همچنین اتصال Agent به ابزارهای MCP آزمایش شد و فراخوانی ابزار توسط مدل با موفقیت انجام گرفت.

در مرحله سوم، Confluence داخلی شرکت با نسخه ۷,۱۵ بررسی شد. دسترسی شبکه از طریق OpenVPN برقرار شد، REST API در دسترس بود و پس از بررسی روش‌های احراز هویت، (PAT) Personal Access Token آزمایش شد. درخواست Python با PAT توانست محتوای واقعی صفحات Confluence را دریافت کند.

نتیجه کلی: مسیر اتصال Llama ۳,۲، LangChain، MCP و Confluence از نظر فنی قابل اجرا است؛ ادامه کار نیازمند تصمیم درباره احراز هویت، سطح دسترسی، حساب سرویس، امنیت Token، TLS و معماری نهایی است.

## ۲. دامنه و وضعیت مراحل انجام شده

| وضعیت    | موضوع                                       | مرحله     |
|----------|---------------------------------------------|-----------|
| موفق     | سرور محلی و File Assistant برای یادگیری MCP | مرحله اول |
| موفق     | Llama 3.2 + Ollama + LangChain              | مرحله دوم |
| PoC موفق | Confluence 7.15 + REST API + احراز هویت     | مرحله سوم |

### ۳. مرحله اول: MCP سرور محلی و ابزارهای File Assistant

در مرحله اول پروژه، هدف اصلی یادگیری عملی معماری MCP (Model Context Protocol) و اثبات ارتباط بین یک Agent و ابزارهای خارجی بود؛ بنابراین یک MCP Server آزمایشی با Python بدون وجود llm ایجاد شد که به عنوان یک Local File Server عمل می کرد. این سرور وظیفه داشت دسترسی کنترل شده به فایل های موجود در یک پوشه مشخص از پروژه را در اختیار Agent قرار دهد. در این مرحله به جای اتصال مستقیم Agent به فایل های سیستم، یک لایه MCP بین Agent و منابع داده قرار گرفت تا ساختار پروژه از همان ابتدا مطابق معماری نهایی طراحی شود.

برای ایجاد سرور از MCPServer استفاده شد.

```
from mcp.server import MCPServer

mcp = MCPServer("Local File Server")
```

در این سرور سه ابزار اصلی پیاده سازی شد: `list_files` : برای دریافت فهرست فایل های موجود در Workspace، `read_file` برای خواندن محتوای یک فایل مشخص و `search_in_files` برای جست و جوی یک عبارت در فایل ها. این تفکیک باعث شد Agent به جای دسترسی مستقیم به فایل های سیستم، عملیات مورد نیاز خود را از طریق Tool های مشخص و قابل کنترل انجام دهد.

به عنوان نمونه، ابزارهای MCP با decorator مربوط به `mcp.tool()` در اختیار Client قرار گرفتند:

```
@mcp.tool()
def list_files() -> list[str]:
    ...
```

در زمان پیاده سازی `read_file`، برای جلوگیری از دسترسی خارج از Workspace، مسیر فایل قبل از خواندن بررسی و به مسیر واقعی تبدیل شد. در صورتی که مسیر نهایی خارج از پوشه مجاز قرار داشته باشد، درخواست با خطای `Access denied` متوقف می شود. این کنترل برای جلوگیری از Path Traversal و محدود کردن سطح دسترسی MCP Server در نظر گرفته شد.

```
file_path = (WORKSPACE / filename).resolve()
if WORKSPACE.resolve() not in file_path.parents:
    raise ValueError("Access denied")
```

### ۳. مرحله دوم: اجرای Llama ۳,۲ به صورت Local

در مرحله دوم پروژه، هدف اصلی آماده سازی مدل زبانی محلی و بررسی امکان استفاده از آن در کنار LangChain بود. برای این منظور ابتدا Ollama روی سیستم نصب شد تا اجرای مدل های زبانی به صورت Local امکان پذیر باشد. پس از نصب Ollama، مدل Llama ۳,۲ دانلود و اجرا شد و در ادامه با استفاده از پکیج langchain-ollama، این مدل به محیط LangChain متصل گردید. این مرحله اهمیت زیادی داشت، زیرا قرار بود مدل زبانی به عنوان هسته تصمیم گیری Agent در معماری پروژه مورد استفاده قرار گیرد و در ادامه همین Agent به ابزارهای MCP متصل شود.

پس از نصب Ollama، اجرای مدل llama۳,۲ به صورت مستقیم بررسی شد تا اطمینان حاصل شود که مدل به درستی در محیط محلی در دسترس است. سپس کلاس ChatOllama از LangChain برای ایجاد رابط بین LangChain و مدل Llama ۳,۲ مورد استفاده قرار گرفت:

```
from langchain_ollama import ChatOllama

llm = ChatOllama(model="llama۳,۲")
```

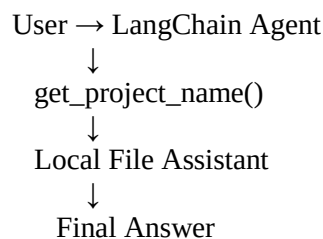
در ادامه، برای بررسی اینکه مدل صرفاً قادر به تولید پاسخ متنی نیست و می تواند در ساختار یک Agent نیز مورد استفاده قرار گیرد، یک Agent ساده با استفاده از قابلیت create\_agent در LangChain ایجاد شد. برای این تست یک Tool بسیار ساده با نام get\_project\_name تعریف شد که وظیفه آن برگرداندن نام پروژه بود. هدف این تست، بررسی ارتباط بین مدل زبانی، Agent و Tool و اطمینان از عملکرد صحیح فرآیند Tool Calling در LangChain بود.

ساخت Agent با مدل Llama ۳,۲ و Tool آزمایشی به صورت زیر انجام شد:

```
agent = create_agent(
    model=llm,
    tools=[get_project_name],
)
```

پس از ایجاد Agent، یک سؤال ساده با مضمون (اسم پروژه چیه؟) به Agent ارسال شد. Agent با توجه به سؤال تشخیص داد که برای پاسخ دقیق باید از Tool مربوط به نام پروژه استفاده کند. Tool اجرا شد و مقدار Local File Assistant را برگرداند؛ سپس Agent بر اساس نتیجه دریافت شده پاسخ نهایی را تولید کرد.

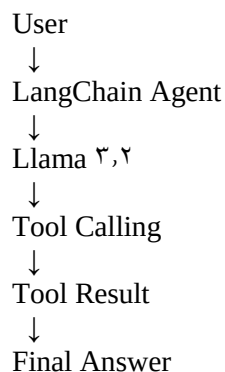
جریان این تست به صورت خلاصه به شکل زیر بود:



این آزمایش نشان داد که Llama ۳,۲ پس از اتصال به LangChain می تواند به عنوان مدل تصمیم گیرنده یک Agent مورد استفاده قرار گیرد و Agent نیز قادر است بر اساس درخواست کاربر، Tool مناسب را انتخاب و نتیجه آن را در پاسخ نهایی استفاده کند. بنابراین در پایان این مرحله، بخش مدل زبانی و Agent پایه پروژه آماده شد و زیرساخت لازم برای اتصال این Agent به Tool های خارجی فراهم گردید.

در ادامه پروژه، همین ساختار پایه به MCP متصل شد؛ به این معنی که به جای یک Tool ساده داخلی، Tool های ارائه شده توسط MCP Server در اختیار Agent قرار گرفتند. در نتیجه معماری از حالت LangChain Agent + Local Tool به سمت LangChain Agent MCP Tools + توسعه پیدا کرد. این تغییر، ارتباط بین مرحله دوم و مرحله اول را ایجاد کرد و زمینه لازم برای استفاده از Llama ۳,۲ در کنار MCP فراهم شد.

معماری این مرحله را می توان به صورت خلاصه چنین نمایش داد:



در مجموع، خروجی مرحله دوم شامل نصب و آماده سازی Ollama، دریافت و اجرای Llama ۳,۲، اتصال مدل به LangChain و ایجاد یک Agent پایه با قابلیت استفاده از Tool بود. موفقیت این مرحله نشان داد که مدل محلی می تواند در معماری Agent پروژه قرار گرفته و در مرحله بعد، همین Agent برای استفاده از قابلیت های MCP توسعه داده شود.

## ۴. مرحله سوم — اتصال به ۷,۱۵ Confluence و پیاده سازی احراز هویت

در مرحله سوم پروژه، هدف اصلی اتصال Agent و زیرساخت MCP به Confluence سازمانی و بررسی امکان دریافت اطلاعات واقعی از صفحات و Space های موجود در آن بود. برخلاف دو مرحله قبلی که تمام اجزای سیستم در محیط Local اجرا می شدند، در این مرحله لازم بود ارتباط با یک سرویس داخلی سازمان برقرار شود؛ بنابراین علاوه بر پیاده سازی نرم افزاری، ابتدا شرایط دسترسی شبکه و نحوه احراز هویت Confluence بررسی شد. سرویس مورد استفاده ۷,۱۵ Confluence بود و دسترسی به آن تنها از طریق شبکه داخلی سازمان امکان پذیر بود.

### ۴-۱. بررسی دسترسی شبکه به Confluence

اولین مرحله برای اتصال به Confluence، فراهم کردن دسترسی شبکه به سرور داخلی بود. برای این منظور ابتدا سیستم به OpenVPN سازمان متصل شد و پس از برقراری VPN، امکان دسترسی به آدرس داخلی Confluence فراهم گردید. این موضوع نشان داد که دسترسی به Confluence صرفاً به داشتن آدرس سرور محدود نیست و سیستم اجراکننده Agent نیز باید از نظر شبکه در محدوده ای قرار داشته باشد که بتواند به سرویس داخلی سازمان دسترسی پیدا کند.

پس از برقراری VPN، آدرس داخلی Confluence در مرورگر Firefox باز شد. سرویس با استفاده از HTTPS در دسترس بود، اما به دلیل گواهی SSL داخلی/غیرقابل اعتماد برای سیستم، مرورگر هشدار Not Secure نمایش داد. برای ادامه تست در محیط داخلی، از بخش Advanced مرورگر گزینه ادامه دادن به سایت انتخاب شد و صفحه ورود Confluence در دسترس قرار گرفت.

پس از عبور از این مرحله، علاوه بر دسترسی شبکه، ورود به خود Confluence نیز مورد نیاز بود. بنابراین دو سطح دسترسی در این مرحله وجود داشت: ابتدا دسترسی شبکه از طریق OpenVPN و سپس احراز هویت در خود Confluence.

### ۴-۲. بررسی ساختار و REST API کانفلوئنس

پس از اطمینان از دسترسی به رابط کاربری Confluence، مرحله بعد بررسی API رسمی این سرویس بود. هدف این بود که مشخص شود آیا اطلاعات Space ها و صفحات Confluence را می توان به صورت برنامه نویسی شده و بدون وابستگی به رابط کاربری مرورگر دریافت کرد یا خیر.

برای این منظور REST API مربوط به Confluence بررسی شد. یکی از Endpoint های مورد استفاده:

/rest/api/space

بود که امکان دریافت اطلاعات مربوط به Space های موجود در Confluence را فراهم می کرد. در زمان تست، Space مربوط به پروژه با کلید AI شناسایی شد و پاسخ API شامل اطلاعاتی مانند نام Space، کلید Space و لینک داخلی آن بود.

نمونه ساختار اطلاعات دریافت شده از API به صورت خلاصه شامل موارد زیر بود:

Space: AI  
Key: AI  
Type: global  
Web UI: /display/AI

این مرحله اهمیت زیادی داشت، زیرا مشخص شد که برای ارتباط برنامه نویسی شده با Confluence نیازی به شبیه سازی رفتار مرورگر نیست و می توان از REST API خود Confluence برای دریافت اطلاعات استفاده کرد.

### ۳-۴. بررسی دسترسی API و مسئله احراز هویت

پس از شناسایی REST API، درخواست ها از محیط Python نیز آزمایش شدند. در این مرحله مشخص شد که صرف داشتن دسترسی شبکه به Confluence برای دریافت اطلاعات کافی نیست و درخواست های API باید همراه با اطلاعات احراز هویت معتبر ارسال شوند.

در ابتدا درخواست API بدون احراز هویت مناسب بررسی شد. نتیجه نشان داد که درخواست Python نمی تواند همان محتوایی را که در مرورگر لاگین شده مشاهده می شود، دریافت کند. بنابراین مشخص شد که Authentication بخش مهم اتصال برنامه به Confluence است و باید مکانیزم احراز هویت مورد استفاده مرورگر یا یک روش رسمی برای دسترسی برنامه ای شناسایی شود.

در همین مرحله، درخواست های ارسال شده توسط مرورگر با استفاده از بخش Network ابزارهای توسعه دهنده Firefox بررسی شدند تا مشخص شود مرورگر چگونه درخواست های احراز هویت شده را به Confluence ارسال می کند.

#### ۴-۴. بررسی Cookie و JSESSIONID

در بررسی Network مرورگر مشخص شد که درخواست‌های معتبر Confluence شامل یک Cookie احراز هویت بودند و در میان اطلاعات Cookie، مقدار JSESSIONID وجود داشت. این مقدار مربوط به Session ایجاد شده پس از ورود کاربر به Confluence بود.

در واقع، مرورگر پس از Login یک Session معتبر ایجاد کرده بود و در درخواست‌های بعدی، شناسه آن Session را به سرور ارسال می‌کرد. بنابراین یکی از روش‌های بررسی شده برای اثبات ارتباط برنامه‌نویسی شده این بود که مقدار Session معتبر مرورگر در یک Python Session قرار داده شود و سپس درخواست API ارسال گردد.

ساختار کلی این تست به شکل زیر بود:

```
session = requests.Session()
session.cookies.set("JSESSIONID", "<session-value>")
response = session.get(url, verify=False)
```

این آزمایش موفق بود و نشان داد که وقتی درخواست Python دارای Session معتبر باشد، Confluence می‌تواند همانند مرورگر پاسخ معتبر API را برگرداند. در نتیجه مشخص شد که مشکل دسترسی API ناشی از خود REST API نیست، بلکه به احراز هویت درخواست مربوط است.

البته JSESSIONID در این مرحله صرفاً به عنوان یک روش بررسی و اثبات ارتباط مورد استفاده قرار گرفت، زیرا این مقدار متعلق به Session یک کاربر در مرورگر است و برای یکپارچه‌سازی پایدار و برنامه‌ای گزینه مناسبی محسوب نمی‌شود.

#### ۴-۵. بررسی Personal Access Token

پس از اثبات اینکه احراز هویت عامل اصلی دسترسی برنامه به Confluence است، روش مناسب‌تر برای ارتباط برنامه‌ای بررسی شد. در پنل پروفایل Confluence، بخش Settings و سپس قسمت Personal Access Tokens شناسایی شد.

در این بخش امکان ایجاد یک Personal Access Token (PAT) برای استفاده برنامه‌ای فراهم بود. با ایجاد Token، برنامه دیگر نیازی به استخراج Session مرورگر یا وابستگی به JSESSIONID نداشت و می‌توانست Token را به عنوان Credential مربوط به درخواست‌های REST API ارسال کند.

پس از ایجاد PAT، اطلاعات مربوط به آن به صورت مستقیم در کد قرار داده نشد و در فایل محیطی env نگهداری شد:

```
CONFLUENCE_BASE_URL=https://X
CONFLUENCE_PAT=YOUR_REAL_TOKEN
```

برای خواندن این مقادیر در Python نیز از python-dotenv استفاده شد:

```
load_dotenv()
BASE_URL = os.getenv("CONFLUENCE_BASE_URL")
PAT = os.getenv("CONFLUENCE_PAT")
```

این ساختار باعث شد اطلاعات حساس مانند Token از سورس کد جدا باشد و در زمان اجرای برنامه از Environment خوانده شود.

#### ۴-۶. ارسال درخواست احراز هویت شده به Confluence

پس از آماده سازی PAT، درخواست REST API با استفاده از Header مربوط به Bearer Token ارسال شد. برای تست اولیه، Endpoint مربوط به محتوای Confluence مورد استفاده قرار گرفت و Space با کلید AI به عنوان محدوده جست و جو مشخص شد.

ساختار اصلی درخواست به این صورت بود:

```
headers = {"Authorization": f"Bearer {PAT}", "Accept": "application/json"}
response = requests.get(url, params=params, headers=headers, verify=False, timeout=۱۰)
```

در این درخواست، پارامتر spaceKey روی AI قرار گرفت و همچنین body.storage برای دریافت محتوای صفحات درخواست شد. در نتیجه، درخواست از حالت صرفاً دریافت Metadata خارج شد و امکان دریافت محتوای واقعی صفحات Confluence نیز مورد بررسی قرار گرفت.

#### ۴-۷. دریافت موفق اطلاعات واقعی از Confluence

نتیجه تست با PAT موفق بود و درخواست Python با ۲۰۰ HTTP Status پاسخ گرفت. پاسخ دریافت شده شامل اطلاعات واقعی مربوط به محتوای Space AI بود و در میان نتایج، صفحه‌ای با عنوان AI Home مشاهده شد.

اطلاعات دریافت شده برای این صفحه شامل شناسه صفحه و محتوای آن در قالب Storage Format بود:

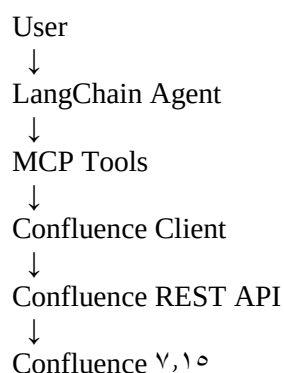
```
Space Key: AI
Page ID: ۴۹۷۴۱۹۱۱
Page Title: AI Home
Content: body.storage
```

این نتیجه نقطه مهم مرحله سوم بود، زیرا برای اولین بار ارتباط برنامه Python با Confluence سازمانی به صورت مستقیم و خارج از مرورگر با استفاده از REST API و Token برقرار شد و برنامه توانست محتوای واقعی موجود در Confluence را دریافت کند.

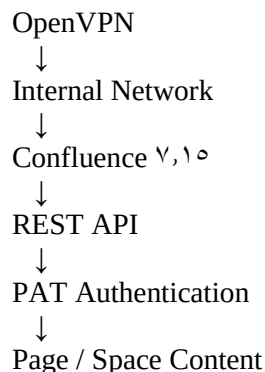
#### ۴-۸. ساختار ارتباط در مرحله سوم

در این مرحله، معماری ارتباط با Confluence به این صورت شکل گرفت که Agent در لایه بالاتر قرار دارد و در نهایت درخواست اطلاعات باید از طریق Tool های MCP به یک Client مربوط به Confluence منتقل شود. در بخش فعلی، تمرکز بر اثبات ارتباط و احراز هویت موفق با خود Confluence بود.

ساختار کلی ارتباط به صورت زیر قابل نمایش است:



در بخش احراز هویت نیز مسیر دسترسی به این شکل بررسی شد:



در مجموع، در مرحله سوم ابتدا دسترسی شبکه‌ای به Confluence از طریق OpenVPN برقرار شد، سپس نحوه ورود و Session مرورگر بررسی گردید و مشخص شد که JSESSIONID در درخواست‌های احراز هویت شده نقش دارد. پس از آن REST API مستقیماً از محیط Python آزمایش شد و با استفاده از Personal Access Token، ارتباط برنامه‌ای با Confluence برقرار گردید. در نهایت درخواست احراز

هویت شده با موفقیت پاسخ ۲۰۰ دریافت کرد و اطلاعات واقعی Space AI و صفحه AI Home با شناسه ۴۹۷۴۱۹۱۱ از Confluence ۷،۱۵ دریافت شد.

بنابراین خروجی اصلی این مرحله، اثبات کامل مسیر `Network Access → Confluence Authentication → REST API → page content` بود و زیرساخت لازم برای استفاده برنامه از اطلاعات Confluence به صورت مستقیم فراهم شد.

## ۵. چالش های موجود در ادامه مسیر

پس از موفقیت در برقراری ارتباط برنامه با ۷،۱۵ Confluence و دریافت محتوای واقعی از Space مربوط به AI، مرحله بعدی پروژه از یک Proof of Concept ساده به سمت یک راهکار واقعی برای جست و جو، بازیابی و پاسخ گویی روی محتوای سازمانی خواهد رفت. در این مرحله، چند موضوع فنی و زیرساختی باید پیش از پیاده سازی نهایی مشخص و طراحی شوند.

### ۵-۱. پیاده سازی MCP Server اختصاصی برای Confluence

در مراحل قبلی، MCP Server برای کار با فایل های Local پیاده سازی شد؛ اما برای سیستم واقعی باید Tool های اختصاصی Confluence طراحی شوند. این Tool ها باید به عنوان تنها مسیر دسترسی Agent به اطلاعات Confluence عمل کنند.

حداقل Tool های مورد نیاز می توانند شامل موارد زیر باشند:

```
search_confluence()
get_page()
get_page_content()
list_spaces()
```

در این معماری، Agent نباید مستقیماً با REST API یا URL های Confluence کار کند، بلکه درخواست خود را از طریق Tool های تعریف شده در MCP ارسال می کند.

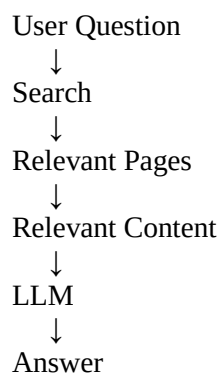
```
Agent
↓
MCP Tool
↓
Confluence Client
↓
Confluence REST API
```

## ۵-۲. طراحی مکانیزم جست و جوی مناسب در Confluence

یکی از مهم ترین نیازهای سیستم واقعی، نحوه پیدا کردن صفحه مرتبط با سؤال کاربر است. دریافت مستقیم صفحات از Confluence به تنهایی کافی نیست؛ زیرا ممکن است تعداد صفحات زیاد باشد و Agent نتواند همه اطلاعات را در Context خود دریافت کند.

بنابراین باید مشخص شود که جست و جوی اولیه چگونه انجام شود؛ برای مثال جست و جوی مستقیم در Confluence، استفاده از قابلیت های جست و جوی خود REST API یا استفاده از ساختارهای Query موجود در Confluence.

هدف این بخش تبدیل درخواست کاربر به یک فرآیند مشخص است:



## ۵-۳. حجم بالای اطلاعات و مسئله Context Window

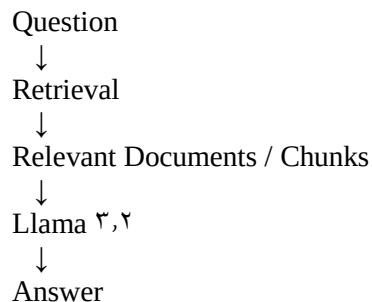
در محیط واقعی احتمالاً تعداد صفحات و حجم محتوای Confluence بسیار بیشتر از تست فعلی خواهد بود. بنابراین نمی توان کل محتوای Space را در هر درخواست به Llama ۳٫۲ ارسال کرد.

برای حل این مسئله باید معماری Retrieval طراحی شود تا فقط بخش های مرتبط با سؤال کاربر در اختیار مدل قرار بگیرد.

در اینجا معماری پروژه از حالت ساده:

Question → Confluence → LLM

به معماری کامل تر زیر تبدیل می شود:

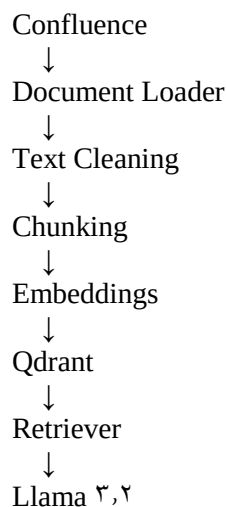


#### ۴-۵. تصمیم گیری درباره استفاده از RAG و Vector Database

در صورت افزایش حجم محتوای Confluence ، احتمالاً نیاز به پیاده سازی یک لایه RAG وجود خواهد داشت. در این حالت صفحات Confluence دریافت، پاک سازی و به بخش های کوچک تر تقسیم می شوند و سپس برای آن ها Embedding تولید می شود.

این Embedding ها می توانند در یک Vector Database مانند Qdrant ذخیره شوند.

معماری احتمالی سیستم در این حالت:



در این مدل، Confluence منبع اصلی دانش و Qdrant لایه بازیابی سریع اطلاعات خواهد بود.

## ۵-۵. تعیین روش بهروزرسانی اطلاعات

در سیستم واقعی باید مشخص شود وقتی یک صفحه در Confluence تغییر می‌کند، چه اتفاقی برای داده ذخیره‌شده در سیستم رخ می‌دهد.

برای مثال اگر یک صفحه امروز وارد سیستم شود و فردا تغییر کند، باید مکانیزمی وجود داشته باشد که نسخه جدید صفحه جایگزین نسخه قبلی شود.

بنابراین باید برای موارد زیر تصمیم‌گیری شود:

- تشخیص صفحات جدید
- تشخیص صفحات تغییر کرده
- حذف یا غیرفعال کردن صفحات حذف‌شده
- بهروزرسانی Embedding ها
- تعیین زمان‌بندی Synchronization

به عبارت دیگر، ارتباط با Confluence نباید فقط یک One-Time Import باشد و باید در صورت نیاز به یک فرآیند Synchronization تبدیل شود.

## ۵-۶. مدیریت دسترسی و Permission های Confluence

یکی از مهم‌ترین مسائل در محیط سازمانی، موضوع سطح دسترسی کاربران است.

در تست فعلی ثابت شد که با یک PAT می‌توان به اطلاعات Confluence دسترسی پیدا کرد؛ اما در سیستم واقعی باید مشخص شود این Token متعلق به چه هویتی است و چه سطح دسترسی‌ای دارد.

برای مثال باید مشخص شود:

User A → چه صفحاتی را می‌تواند ببیند؟

User B → چه صفحاتی را می‌تواند ببیند؟

Agent → دسترسی دارد؟ Confluence با چه هویتی به

اگر تمام کاربران از یک PAT مشترک استفاده کنند، باید بررسی شود که آیا این مدل با سیاست امنیتی سازمان سازگار است یا خیر.

در نتیجه تصمیم درباره User-based Authentication، Service Account یا یک مدل Delegated Access یکی از تصمیمات مهم معماری خواهد بود.

## ۵-۷. مدیریت امن Token و اطلاعات حساس

در PAT، PoC، env در قرار گرفت که برای محیط توسعه مناسب است، اما برای محیط Production باید روش امن تری برای نگهداری Secret ها در نظر گرفته شود.

در محیط واقعی باید موارد زیر مشخص شود:

- محل نگهداری Token
- نحوه Rotation Token
- زمان اعتبار Token
- سطح دسترسی Token
- نحوه جلوگیری از ثبت Token در Log
- نحوه مدیریت Secret در Deployment

به خصوص نباید Token در Source Code یا Repository قرار گیرد.

## ۵-۸. مدیریت SSL و Certificate

در مرحله تست، برای ارتباط Python از `verify=False` استفاده شد تا ارتباط با سرور داخلی و API بررسی شود. این گزینه صرفاً برای تست اولیه مناسب است و در محیط Production نباید به عنوان راهکار نهایی استفاده شود.

در محیط واقعی باید Certificate داخلی سازمان یا CA مورد اعتماد در محیط اجرای برنامه نصب و Trust شود تا ارتباط HTTPS به شکل استاندارد برقرار شود.

هدف نهایی باید این باشد:

```

Python
↓HTTPS
Trusted Internal CA
↓
Confluence

```

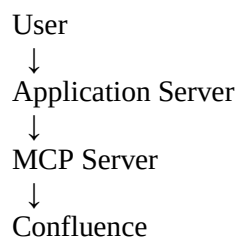
و نه استفاده دائمی از:

verify=False

## ۵-۹. محل اجرای نهایی سیستم

از آنجا که Confluence داخل شبکه سازمان قرار دارد، محل اجرای Agent و MCP Server اهمیت زیادی پیدا می کند.

سیستمی که قرار است در Production اجرا شود باید بتواند به Confluence دسترسی شبکه ای داشته باشد. بنابراین باید مشخص شود راهکار نهایی روی چه زیرساختی اجرا خواهد شد؛ برای مثال:

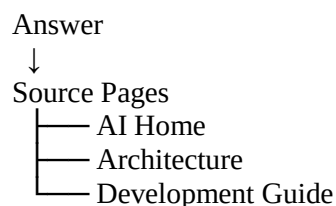


و مشخص شود آیا سرور موردنظر مستقیماً به شبکه داخلی دسترسی دارد یا نیازمند VPN ، Route یا تنظیمات شبکه ای دیگری است.

## ۵-۱۰. طراحی پاسخ گویی مبتنی بر Source

در یک سیستم سازمانی بهتر است Agent صرفاً پاسخ متنی تولید نکند، بلکه مشخص کند پاسخ بر اساس کدام صفحه یا صفحات Confluence تولید شده است.

برای مثال:



این موضوع باعث می شود کاربر بتواند منبع پاسخ را بررسی کند و اعتمادپذیری سیستم افزایش پیدا کند.

## ۱۱-۵. کنترل کیفیت پاسخ های Llama ۳,۲

حتی اگر Retrieval درست انجام شود، مدل زبانی ممکن است در تفسیر اطلاعات یا تولید پاسخ دچار خطا شود. بنابراین در مرحله بعد باید مشخص شود Agent در شرایطی که اطلاعات کافی در Confluence پیدا نمی کند چه رفتاری داشته باشد.

برای مثال نباید در صورت نبود اطلاعات معتبر، پاسخ ساختگی تولید کند.

رفتار مورد انتظار می تواند این باشد:

Relevant Information Found



Generate Answer

No Reliable Information



"No sufficient information found in Confluence."

بنابراین باید Prompt ، رفتار Agent و معیارهای ارزیابی پاسخ ها طراحی و تست شوند.

## ۱۲-۵. cache و performance

در محیط واقعی، هر سؤال کاربر نباید الزاماً باعث یک درخواست کامل و سنگین به Confluence شود. در صورت افزایش تعداد کاربران یا درخواست ها، زمان پاسخ و تعداد درخواست های API اهمیت پیدا می کند.

بنابراین در مراحل بعد باید مواردی مانند:

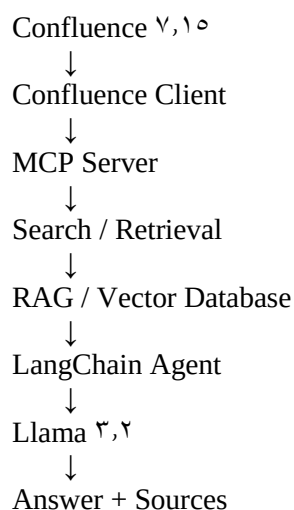
- Cache
- Timeout
- Connection Management
- تعداد درخواست های هم زمان
- Pagination
- محدود کردن حجم پاسخ API

بررسی شوند.

## جمع بندی نیازمندی های ادامه پروژه

با توجه به تست های انجام شده، اتصال پایه به Confluence با موفقیت اثبات شده است؛ بنابراین بخش بعدی پروژه دیگر صرفاً مسئله اتصال به Confluence نیست، بلکه مسئله اصلی تبدیل این اتصال به یک سیستم قابل استفاده برای جست و جو و پاسخ گویی روی دانش سازمانی است.

مسیر پیشنهادی توسعه به صورت کلی چنین خواهد بود:



برای ورود به مرحله اجرایی بعدی، مهم ترین تصمیمات مورد نیاز شامل طراحی Tool های MCP برای Confluence، تعیین مدل جست و جو و Retrieval، تصمیم درباره استفاده از RAG و Qdrant، تعیین مدل احراز هویت و Permission کاربران، طراحی Synchronization اطلاعات و مشخص کردن زیرساخت اجرای Production خواهد بود.

## ۶. واژه نامه سریع

| اصطلاح       | معنی کوتاه                                                           |
|--------------|----------------------------------------------------------------------|
| MCP          | پروتکل زمینه مدل                                                     |
| MCP server   | سروری که ابزار و داده را به عامل هوشمند ارائه می دهد.                |
| MCP client   | واسطی که بین عامل و سرور ارتباط برقرار می کند                        |
| Ollam        | ابزار متن باز برای اجرای محلی مدل های زبانی بزرگ                     |
| Llama ۳,۲    | مدل زبانی بزرگ مناسب برای اجرای محلی                                 |
| Langchain    | فریمورک پایتون برای ساخت عامل های هوشمند                             |
| Tool Calling | قابلیتی که به عامل اجازه می دهد ابزارها یا توابع خاص را فراخوانی کند |
| RAG          | بازیابی دانش مرتبط و افزودن آن به Context مدل                        |